Cloud-Based Solution for Migrating a Legacy University Management System

TASK 1: Introduction to Cloud Computing and Mapping to the University Scenario

1.1 Definition and Five Essential Characteristics

Cloud computing is a model for delivering computing resources (servers, storage, databases, networking, software and analytics) over the internet on a pay-as-you-use basis. The standard NIST definition describes it as a model for enabling convenient, on-demand network access to a shared pool of configurable computing resources that can be rapidly provisioned and released with minimal management effort or provider interaction.

Table 1: Five Essential Characteristics of Cloud Computing and Their Meaning for the University

Characteristic	**Meaning**	**Relevance to the University**
On-demand self-service	Users provision servers, storage or applications themselves through a portal or API, without contacting the provider.	The IT team can create a new server in minutes instead of waiting 6 to 8 weeks for procurement.
Broad network access	Services are available over the network from phones, laptops and tablets using standard protocols.	Students and staff on all 4 campuses, and later the mobile app and partner colleges, can reach the services from anywhere.
Resource pooling	The provider serves many customers from a shared pool of resources using multi-tenancy, assigning and reassigning resources dynamically.	The 12 idle servers (15 percent utilisation) are replaced by a shared pool, so capacity is not wasted.
Rapid elasticity	Capacity can be scaled out or in quickly, often automatically, so resources appear unlimited.	Extra capacity is added during admissions, result publication and exam weeks, and released afterwards.
Measured service	Usage is metered, monitored and billed, giving transparency for both the provider and the customer.	The university pays only for what it uses, and departments can be charged by usage.

1.2 Deployment Models and the Most Suitable Model

Table 2: Comparison of Cloud Deployment Models

Model	Description	Advantages	Limitations
Public cloud	Owned and operated by a provider, shared by many customers over the internet (AWS, Azure, Google Cloud).	No capital cost, very high elasticity, global reach.	Less direct control, data location and compliance concerns.
Private cloud	Infrastructure used by a single organisation, on-premises or hosted.	Maximum control and security.	High capital cost, limited elasticity, needs skilled staff.
Hybrid cloud	A combination of private and public clouds connected securely, allowing workloads to move between them.	Balances control and elasticity, supports gradual migration.	More complex networking, security and management.
Community cloud	Shared by several organisations with common needs (for example, universities and research bodies).	Shared cost and shared compliance requirements.	Limited scale, governance must be agreed between members.

Recommended model: hybrid cloud with a public-cloud-first approach. Elastic workloads such as the web portals, exam portal, LMS and email run in the public cloud so that they can scale for peak weeks. Sensitive records and the legacy database remain in a private, on-premises environment during migration and move to a managed in-country cloud database later, once security and compliance are confirmed. A secure VPN or private link connects the two. This model suits the university because it allows phased migration, protects student data and provides elasticity at a controlled cost. If the affiliated colleges later share services, a community-cloud style arrangement can be layered on top.

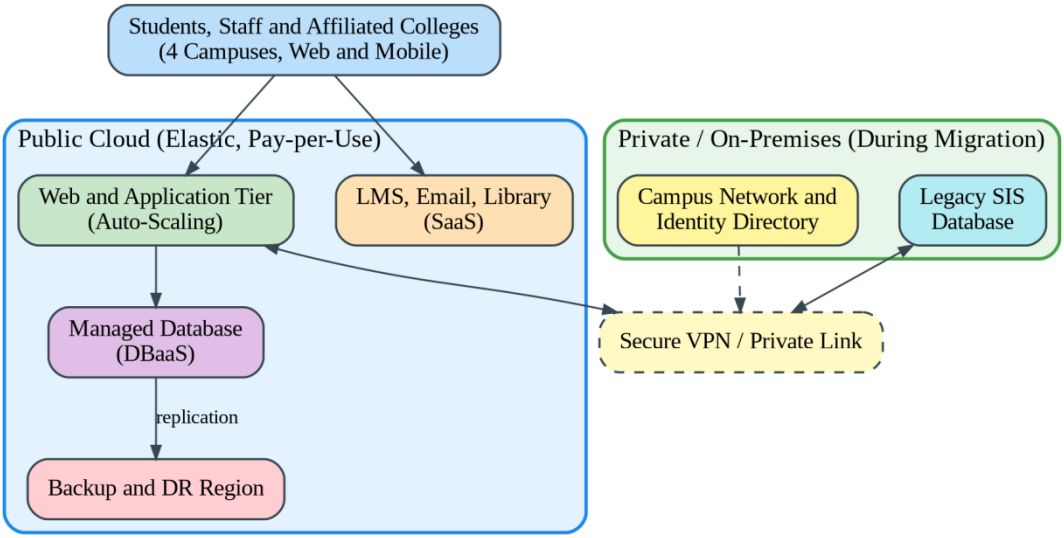


Figure 1: Hybrid Cloud Deployment Model Proposed for the University

1.3 Three Problems of the Current Setup and How Cloud Characteristics Address Them

Table 3: Mapping Current Problems to Cloud Characteristics

Problem in the On-Premise Setup	**Cloud Characteristic That Addresses It**	**How the Problem Is Solved**
Servers idle at about 15 percent utilisation, while capital is locked in hardware.	Resource pooling and measured service	Workloads share pooled resources and are consolidated, and the university pays only for actual consumption.
Systems fail to cope with peaks above 95 percent during admissions, results and exams.	Rapid elasticity	Auto-scaling adds servers when load rises and removes them afterwards, so peaks are served without permanent over-provisioning.
Slow procurement of 6 to 8 weeks per server, and no disaster recovery site.	On-demand self-service and broad network access	Servers are provisioned in minutes, and a backup or recovery environment in another region can be created and reached over the network without building a second data centre.

1.4 Benefits and Limitations of Cloud Adoption

Table 4: Benefits and Limitations of Cloud Adoption for the University

Benefits	**Limitations**
Lower capital expenditure and conversion to operating expense.	Recurring costs can grow if usage is not monitored.
Elastic scaling for admissions and exam peaks.	Dependence on internet connectivity and the provider.
Built-in disaster recovery and high availability.	Data privacy, data residency and compliance responsibilities.
Faster deployment and easier mobile and partner access.	Vendor lock-in and migration effort for monolithic applications.
Less hardware maintenance, so staff can focus on services.	Skills gap: staff need training in cloud management and security.

TASK 2: Evolution of Computing Infrastructure and Hardware Evolution

2.1 Timeline from Mainframes to Cloud

Computing infrastructure evolved in stages. Each stage solved a limitation of the stage before it, and together they created the conditions that made cloud computing possible. Figure 2 shows the progression.

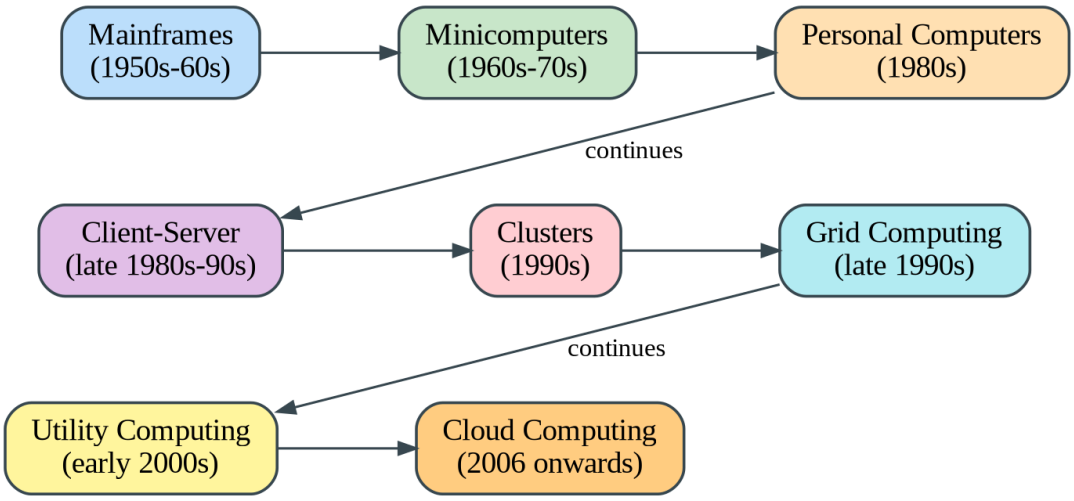


Figure 2: Evolution of Computing Infrastructure from Mainframes to Cloud Computing

Table 5: Stages of Evolution and the Limitation Each Stage Solved

Stage	**Period**	**Key Idea**	**Limitation It Solved / Remaining Limitation**
Mainframes	1950s to 1960s	Large central computers shared by many users through terminals.	Solved manual batch computing, but they were extremely expensive, and only large organisations could own them.
Minicomputers	1960s to 1970s	Smaller, cheaper machines for departments.	Reduced cost and brought computing to departments, but they were still isolated and limited in power.
Personal computers	1980s	Low-cost computers on every desk.	Gave users independence, but data and software were scattered and hard to share and manage.

Stage	Period	Key Idea	Limitation It Solved / Remaining Limitation
Client-server	Late 1980s to 1990s	Clients request services from central servers over a network.	Centralised data and applications, but each application often had its own dedicated server, and scaling meant buying more hardware.
Clusters	1990s	Groups of connected computers working as a single system.	Provided higher performance and availability from commodity machines, but within one organisation and one location.
Grid computing	Late 1990s	Resources from many locations combined to solve large problems.	Enabled large-scale sharing across organisations, but it was complex and focused on scientific computing.
Utility computing	Early 2000s	Computing offered as a metered utility, like electricity.	Introduced pay-per-use billing, but needed mature virtualisation and fast networks to deliver.
Cloud computing	2006 onwards	Virtualised, elastic, self-service resources delivered over the internet.	Combines virtualisation, pooling, elasticity and metered billing, so organisations no longer need to own infrastructure.

2.2 Hardware Evolution That Enabled the Cloud

- **Multi-core processors:** Several processing cores on one chip, with hardware support for virtualisation (Intel VT-x, AMD-V), allow many virtual machines to run efficiently on one physical server.
- **Commodity servers:** Standard, low-cost x86 servers, built in very large numbers and designed to be replaced rather than repaired, made massive data centres affordable.
- **High-capacity storage:** Dense hard disks, solid-state drives and distributed storage systems provide cheap, durable and fast storage at petabyte scale.
- **High-speed networking:** Gigabit and 10, 40 and 100 gigabit Ethernet and broadband internet connect data centres and users with low latency, which makes remote use of resources practical.

- **Data centres:** Large facilities with efficient power and cooling, redundancy and automation operate thousands of servers, giving providers economies of scale that individual organisations cannot match.

Together, these advances meant that powerful, reliable and cheap computing could be pooled in large facilities and reached from anywhere. Virtualisation software then divided this hardware into flexible units that customers could rent.

2.3 Stage of the University's Current System

The university's current environment corresponds to the **client-server stage** in its dedicated-server form. The reasons are as follows.

- Each application (Student Information System, LMS, Library, Exam Portal and Email) runs on its own dedicated physical server in a single on-premise data centre, which is the typical client-server pattern.
- Users on the 4 campuses access the applications as clients over the campus network.
- There is no virtualisation, resource sharing or elasticity: servers average 15 percent utilisation but reach more than 95 percent during peaks, which shows that capacity cannot be shared or scaled.
- Capacity is added by buying hardware (6 to 8 weeks per server), and there is no disaster recovery site, which are limitations that utility computing and cloud computing were designed to remove.

TASK 3: Internet Software Evolution

3.1 Evolution of Internet Software

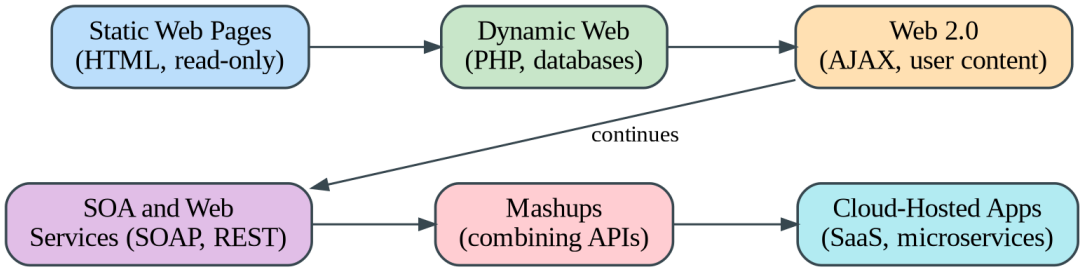


Figure 3: Evolution of Internet Software from Static Pages to Cloud-Hosted Applications

Table 6: Stages of Internet Software Evolution

Stage	**Description**	**Importance for Cloud**
Static web pages	Fixed HTML documents delivered by a web server for reading only.	Established the web, HTTP and browsers as a universal interface.
Dynamic web	Pages generated on the server using scripts (CGI, PHP, ASP) and databases.	Made online applications such as result lookup and admissions forms possible.
Web 2.0	Interactive, user-driven sites using AJAX, with user-generated content, social features and rich interfaces.	Showed that full applications could run in a browser without installation.
Service-Oriented Architecture (SOA)	Applications built as loosely coupled services with standard interfaces (SOAP, REST).	Allowed software functions to be reused and called by other systems over the network.
Mashups	New applications that combine data and functions from several web APIs (for example, a map plus a timetable).	Demonstrated the value of open APIs and composition of services.
Cloud-hosted applications	Software delivered as a service from cloud platforms, often built as microservices and used through browsers and mobile apps.	Software is now consumed as a service, with elastic scaling and subscription pricing.

3.2 Role of Browsers, HTTP, AJAX and APIs

- **Browsers:** Act as a universal client, so users need no installed software and applications run on any device.
- **HTTP/HTTPS:** The standard protocol for requests and responses between clients and servers. HTTPS adds encryption, and its simple, stateless design allows servers to be scaled behind load balancers.
- **AJAX:** Allows a page to exchange data with the server in the background without reloading, giving fast, desktop-like interfaces and reducing the data transferred.
- **APIs:** Define how software components communicate (for example, REST APIs returning JSON). They let mobile apps, partner systems and other services use the university's functions, and they are the foundation of cloud services.

3.3 Comparison of Desktop, Client-Server and Web-Delivered Software

Table 7: Comparison of Software Delivery Models

Aspect	**Desktop Software**	**Client-Server Software**	**Web-Delivered (Cloud) Software**
Installation	Installed on every computer.	Client program installed on each machine, server maintained centrally.	No installation; accessed through a browser or app.
Updates	Manual on each machine.	Client updates needed for each user.	Updated once on the server; all users get the new version.
Data location	Local to the computer.	Central server.	Central, in the cloud, accessible anywhere.
Scalability	Not applicable, each PC works alone.	Limited by the server hardware.	Elastic, scaled automatically.
Platform dependence	Tied to an operating system.	Often tied to specific client platforms.	Platform independent.
Cost model	One-time licence per machine.	Licence plus server cost.	Subscription or pay-per-use.
Availability	Works offline, but data is not shared.	Needs network access to the server.	Needs internet but supports access from anywhere.

3.4 Effect on the Design of the University's Applications

The university's applications are large monolithic programs with no public interfaces. The evolution of internet software shows that they should be redesigned or wrapped so that they behave like modern web-delivered services.

- **API-first design:** Each function (results, fees, library search) should be exposed through REST APIs, so the planned mobile app and partner colleges can use them.
- **Modular or service-based structure:** The monolith should be gradually divided into independent services that can be updated and scaled separately.
- **Browser and mobile-friendly interfaces:** Responsive web interfaces replace thick clients, so students need no installation.
- **Stateless application tier:** Session data is kept in a cache or database, so any server can handle any request and auto-scaling works.
- **Standard protocols and security:** HTTPS, token-based authentication and standard formats (JSON/XML) support integration and protect data.
- **Cloud delivery:** Standard functions such as email, LMS and library can be consumed as SaaS rather than maintained locally.

TASK 4: Applying Server Virtualization

4.1 How a Hypervisor Works, and Type-1 versus Type-2 Hypervisors

Virtualization creates multiple isolated virtual machines (VMs) on a single physical server. The **hypervisor** (virtual machine monitor) is the software layer that makes this possible. It abstracts the physical CPU, memory, storage and network and divides them into virtual hardware, allocates it to each VM, schedules VM access to the processor, and isolates the VMs from each other. Each VM runs its own guest operating system and applications as if it had its own computer. Modern CPUs provide hardware assistance (Intel VT-x, AMD-V) to make this efficient.

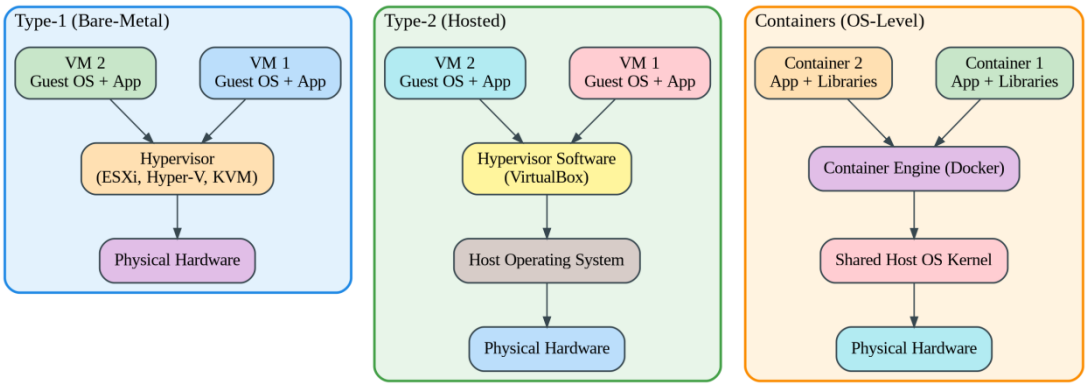


Figure 4: Type-1 Hypervisor, Type-2 Hypervisor and Container Architectures

Table 8: Comparison of Type-1 and Type-2 Hypervisors

Aspect	**Type-1 (Bare-Metal)**	**Type-2 (Hosted)**
Runs on	Directly on physical hardware.	On top of a host operating system.
Performance	High, with little overhead.	Lower, because of the host OS layer.
Typical use	Data centres and cloud platforms for production workloads.	Desktops, learning and testing.
Examples	VMware ESXi, Microsoft Hyper-V, KVM, Xen.	Oracle VirtualBox, VMware Workstation, Parallels.
Security and stability	Smaller attack surface and more stable.	Depends on the security of the host OS.

4.2 Types of Virtualization

Table 9: Comparison of Virtualization Techniques

Technique	How It Works	Advantages	Limitations
Full virtualization	The hypervisor fully emulates hardware, so an unmodified guest OS runs without knowing it is virtual (with binary translation or hardware assistance).	Runs any OS; strong isolation.	Some overhead without hardware assistance.
Para-virtualization	The guest OS is modified to call the hypervisor directly (hypercalls) instead of using emulated hardware.	Better performance for I/O.	Needs a modified guest OS.
OS-level virtualization (containers)	Containers share the host kernel, isolated using namespaces and control groups.	Lightweight, fast start in seconds, high density.	Same OS kernel required; weaker isolation than VMs.
Hardware-assisted virtualization	CPU extensions (VT-x, AMD-V) let the processor handle virtualization instructions.	Near-native speed; simplifies hypervisors.	Requires supporting hardware, and BIOS setting must be enabled.

4.3 Which Applications Should Run as Virtual Machines and Which as Containers

Table 10: Proposed Placement of University Applications

Application	Runs As	Reason
Student Information System (SIS)	Virtual machine now; containers after refactoring	Large monolithic application with a database and strict data protection needs, so a VM gives strong isolation and needs no code changes. It can be broken into containerised services later.
Learning Management System (LMS)	Virtual machine (or SaaS)	Typically packaged software with several components (web server, database, file storage) that expects a full OS. Isolation and compatibility are simpler on a VM.

Application	Runs As	Reason
Library System	Container	Small, lightly loaded, stateless web application. Containers use fewer resources and are quick to deploy and update.
Exam Portal	Container (with auto-scaling)	Heavy, short-lived peaks need rapid scale-out. Containers start within seconds and allow many instances per host.
Email	Virtual machine (or SaaS)	Stateful, mail-server software with disk-intensive storage and long-term operation, which suits a VM. Migrating to a SaaS email service is preferred.

4.4 Consolidation Calculation

Given: 12 physical servers, average CPU utilisation of 15 percent each, target utilisation of the new virtualization host of 70 percent.

Assumptions:

- Each new host has the same processing capacity as one existing server (this is conservative, since modern servers are normally more powerful).
- CPU is the limiting resource for this estimate. Memory, storage and network are checked separately when sizing.
- The 15 percent average is spread evenly, and virtualization overhead is small and covered by the 70 percent target.

Step 1: Total useful workload in server-equivalents

```
Total load = number of servers x average utilisation
Total load = 12 x 15% = 12 x 0.15 = 1.8 server-equivalents
```

Step 2: Hosts required at the 70 percent target

```
Hosts required = total load / target host utilisation
Hosts required = 1.8 / 0.70 = 2.57, which is rounded up to 3 hosts
```

Step 3: Add redundancy (N+1)

A virtualized environment must continue to run if one host fails or is under maintenance (and to allow live migration). One extra host is therefore added: 3 + 1 = **4 hosts**.

Step 4: Consolidation result

```
Consolidation ratio = 12 servers : 4 hosts = 3 : 1
Reduction in physical servers = (12 - 4) / 12 = 66.7%
```

Capacity with a more powerful host: If each new host has twice the capacity of an old server, hosts required = 1.8 / (2 x 0.70) = 1.29, rounded up to 2, plus 1 for N+1 gives 3 hosts. The estimate of 4 hosts is therefore safe.

Peak-load caution: The average of 15 percent hides the peaks. Suppose that during admissions and exam weeks the SIS, LMS, Exam Portal and Email servers reach 95 percent at the same time while the other 8 stay at 15 percent. The load is 4 x 0.95 + 8 x 0.15 = 5.0 server-equivalents, and 5.0 / 0.70 = 7.1, which needs about 8 hosts. Buying 8 hosts for a few weeks would again waste capital. The design therefore keeps 4 hosts for the normal load and uses cloud auto-scaling (bursting) to add temporary capacity for peaks. The result should be confirmed with load testing and measured memory and storage usage before purchase or contract.

4.5 Benefits and Risks of Virtualization

- **Live migration:** A running VM is moved from one host to another with almost no interruption (vMotion, Hyper-V Live Migration). This allows hardware maintenance and load balancing without downtime.
- **Snapshots:** The state of a VM is saved at a point in time, so it can be restored quickly after a failed update or error, and used for backup and testing.
- **Resource over-commitment:** More virtual CPU and memory is assigned than physically exists, on the assumption that VMs rarely use their full allocation at once. This improves utilisation and reduces cost, but must be controlled.

Table 11: Risks of Virtualization and Their Mitigation

Risk	Explanation	Mitigation
Noisy neighbour	One VM consumes excessive CPU, memory or I/O and slows the others on the same host.	Resource limits and reservations, monitoring, and moving busy VMs to other hosts.
Hypervisor failure	If the hypervisor or host fails, all VMs on it stop. The hypervisor is a single point of failure.	Clusters with N+1 hosts, automatic VM restart, regular patching and backups.

Risk	Explanation	Mitigation
Security isolation	A flaw in the hypervisor or a misconfiguration could allow a VM escape or data leakage between VMs.	Patch regularly, harden the hypervisor, segment networks, restrict management access, and separate sensitive VMs.
Over-commitment risk	Excessive over-commitment causes swapping and slow performance during peaks.	Conservative ratios and capacity monitoring.
VM sprawl	Easy creation leads to unmanaged, unused VMs.	Governance, tagging and regular clean-up.

TASK 5: Applying Web Services

5.1 Overview of Web Services in Cloud and SOA

A **web service** is a software function made available over a network using standard web protocols, so that different applications, written in different languages and running on different platforms, can use it. In **Service-Oriented Architecture (SOA)**, business functions are packaged as independent, reusable services with well-defined interfaces. In the cloud, web services are the way users, mobile apps and other systems consume cloud applications and the way cloud components communicate. For the university, they allow result lookup, fee payment and library search to be offered to the mobile app and to partner colleges without exposing the internal systems.

5.2 SOAP, WSDL, UDDI and RESTful Services

Table 12: Comparison of SOAP, WSDL, UDDI and REST

Item	Description	Key Points
SOAP	Simple Object Access Protocol: a protocol that sends XML messages (envelope, header, body) over HTTP, SMTP and others.	Strict standards, built-in error handling, WS-Security and transaction support. Heavier and more verbose.
WSDL	Web Services Description Language: an XML document describing a service's operations, messages and endpoint.	Provides a formal contract. Tools can generate client code.
UDDI	Universal Description, Discovery and Integration: a registry where services are published and found.	Intended for service discovery; rarely used in modern systems.
REST	Representational State Transfer: an architectural style that exposes resources through URLs and uses HTTP methods (GET, POST, PUT, DELETE).	Lightweight and stateless, commonly uses JSON (or XML), is cacheable and easy to use from mobile and web apps.

Table 13: SOAP versus REST

Aspect	SOAP	REST
Format	XML only.	JSON, XML, text and others (JSON is most common).
Style	Protocol with strict rules.	Architectural style with flexible rules.
Operations	Defined in WSDL and sent as POST messages.	HTTP methods on resource URLs.
Performance	Heavier messages, slower.	Lightweight and faster, and responses can be cached.
Security	WS-Security for message-level security.	HTTPS with OAuth2 or API keys and tokens.
Best for	Enterprise integration needing formal contracts and transactions.	Mobile apps, web apps and public APIs.

5.3 Three University Functions Exposed as Web Services

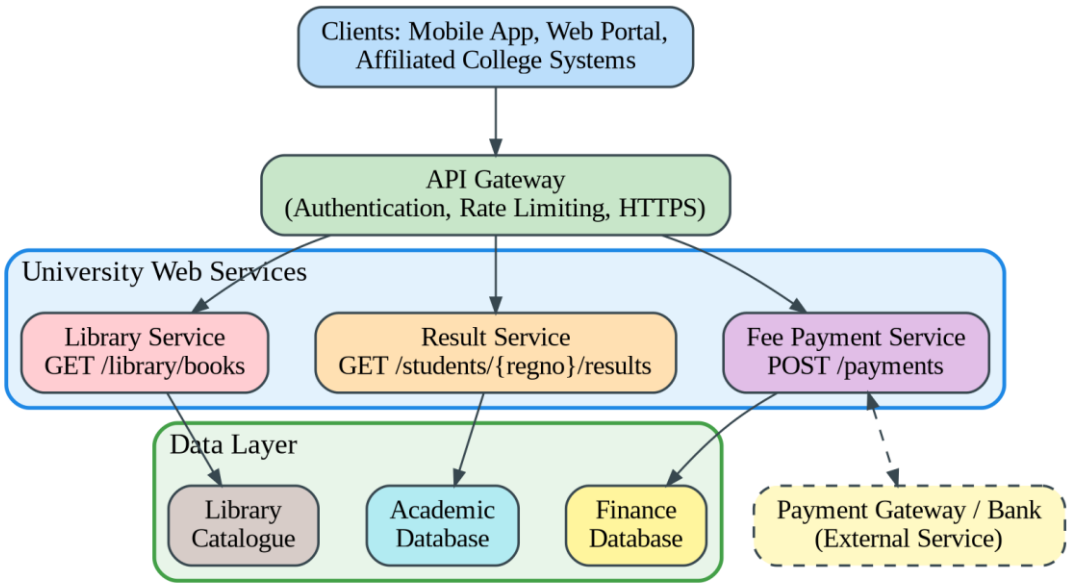


Figure 5: Web Services Layer Exposing University Functions through an API Gateway

Service 1: Result Lookup

Table 14: Result Lookup API Definition

Item	Definition
Endpoint	/api/v1/students/{regno}/results
Method	GET
Sample request	GET /api/v1/students/SIMATS1001/results?semester=5 (Header: Authorization: Bearer <token>)
Sample response (HTTP 200)	{ "regno": "SIMATS1001", "name": "Anitha R", "semester": 5, "cgpa": 8.7, "status": "Pass" }
Error response (HTTP 404)	{ "error": "Student not found", "regno": "XYZ999" }

Service 2: Fee Payment

Table 15: Fee Payment API Definition

Item	Definition
Endpoint	/api/v1/payments
Method	POST
Sample request	POST /api/v1/payments Body: { "regno": "SIMATS1001", "feeType": "Tuition", "amount": 45000, "currency": "INR", "method": "UPI" }
Sample response (HTTP 201)	{ "transactionId": "TXN20261101001", "status": "SUCCESS", "amount": 45000, "receiptUrl": "/api/v1/receipts/TXN20261101001" }
Error response (HTTP 400)	{ "error": "Invalid amount" }

Service 3: Library Search

Table 16: Library Search API Definition

Item	Definition
Endpoint	/api/v1/library/books
Method	GET
Sample request	GET /api/v1/library/books?title=cloud+computing

Item	Definition
Sample response (HTTP 200)	{ "count": 2, "books": [{ "isbn": "9780133544534", "title": "Cloud Computing: Concepts, Technology and Architecture", "available": 3 }, { "isbn": "9780134682433", "title": "Cloud Computing: Theory and Practice", "available": 0 }] }

5.4 Justification of SOAP or REST for Each Function

Table 17: Choice of SOAP or REST for Each Service

Function	Choice	Justification
Result lookup	REST	A simple read-only query used mainly by browsers and the mobile app. JSON is light, the responses can be cached, and no complex transaction is needed.
Fee payment	REST (with strong security)	The mobile app and the 40 colleges need a lightweight, widely supported interface. Security is provided with HTTPS, OAuth2 tokens, request signing and idempotency keys to prevent duplicate payments. If a legacy bank gateway requires SOAP with WS-Security, a SOAP adapter is used only for that back-end link.
Library search	REST	Read-only searches with filters in the URL. Results can be cached to reduce load, and the format is easy for any client.

Overall, REST is preferred for all three functions because the university needs to serve mobile and partner clients quickly, and it needs simple, scalable and cacheable interfaces. SOAP remains an option only where a formal contract and message-level security are mandated by an external system.

TASK 6: Analysis of the Service Models: IaaS, PaaS and SaaS

6.1 Definitions and Responsibility Layers

- **Infrastructure as a Service (IaaS):** The provider supplies virtualised computing infrastructure (virtual machines, storage and networks). The customer installs and manages the operating system, middleware, runtime and applications.
- **Platform as a Service (PaaS):** The provider supplies a ready platform (operating system, runtime, middleware, and often databases and scaling) for building and running applications. The customer manages only the application and its data.
- **Software as a Service (SaaS):** The provider delivers a complete application over the internet. The customer uses it and manages only users, settings and content.

Table 18: Responsibility Layers in On-Premises, IaaS, PaaS and SaaS (Y = Managed by the Customer, P = Managed by the Provider)

Layer (Top to Bottom)	**On-Premises**	**IaaS**	**PaaS**	**SaaS**
Application	Y	Y	Y	P
Data	Y	Y	Y	Y (own content and settings)
Runtime	Y	Y	P	P
Middleware	Y	Y	P	P
Operating system	Y	Y	P	P
Virtualization	Y	P	P	P
Servers	Y	P	P	P
Storage	Y	P	P	P
Networking	Y	P	P	P

Moving from IaaS to PaaS to SaaS, the customer gains convenience and a lower management burden but loses control and customisation.

6.2 Categorisation of the Five University Applications

Table 19: Mapping of University Applications to Service Models

Application	Service Model	Reason
Student Information System (SIS)	PaaS	Custom, data-intensive application. It can be re-hosted on a managed application platform with a managed database, so the university manages only the application and data, while the provider handles patching, scaling and availability.
Learning Management System (LMS)	SaaS	Standard functionality (courses, assignments, grades, content) is available in mature products such as Moodle Cloud, Canvas or Google Classroom. Using SaaS removes server management and gives quick scaling.
Library System	SaaS	Library management is a standard, low-customisation function offered by hosted services. The workload is small, so the effort of running it locally is not justified.
Exam Portal	IaaS	Needs fine control over compute, networking and security (for example, proctoring and secure browser integration) and very large, short peaks. Virtual machines or containers with auto-scaling provide the control and elasticity, and the university keeps the application logic.
Email	SaaS	A commodity service with no university-specific logic. Services such as Microsoft 365 or Google Workspace provide high availability, spam protection and storage, and are often offered at education prices.

Reasoning behind each decision:

- **SIS to PaaS:** The SIS contains the university's own business rules, so it cannot simply be replaced by a standard product. It also does not need operating-system level control, so IaaS would add unnecessary administration. PaaS strikes the balance: custom code runs on a managed platform, and a managed database simplifies backup and scaling.
- **LMS to SaaS:** Teaching and learning features are standard, and fast delivery and ease of use for 25,000 students matter more than deep customisation. Integration with the SIS is done through APIs and single sign-on.

- **Library to SaaS:** It is a low-risk, low-load function. Using SaaS frees the staff and servers at the lowest cost, and it is a good candidate for an early migration.
- **Exam Portal to IaaS:** It is the most time-critical application. Strong security controls, predictable behaviour under load and flexible configuration (such as dedicated networks and special software) are needed, which IaaS provides. Auto-scaling groups give the elasticity for exam weeks.
- **Email to SaaS:** Running mail servers gives little advantage. SaaS gives better security, uptime and storage with no maintenance.

6.3 Comparison of the Models

Table 20: Comparison of IaaS, PaaS and SaaS

Criterion	**IaaS**	**PaaS**	**SaaS**
Control	High: OS, middleware and applications.	Medium: application and data only.	Low: configuration only.
Cost	Pay for resources; moderate cost but needs more staff.	Pay for platform use; lower staff cost.	Subscription per user; predictable, lowest effort.
Scalability	Scalable, but the customer must configure auto-scaling.	Often automatic and built in.	Handled entirely by the provider.
Maintenance effort	High: patching, OS and software management.	Low to medium: application maintenance only.	Very low.
Vendor lock-in	Low to medium: VMs are fairly portable.	Medium to high: platform-specific services.	High: data and workflow tied to the product.
Customisation	Very high.	High within platform limits.	Low.
Best suited for	Custom, specialised or legacy workloads.	Application development and deployment.	Standard business functions.

6.4 Real Provider Examples

Table 21: Provider Examples for Each Service Model

Model	**Example Provider and Product**
IaaS	Amazon Web Services EC2, Microsoft Azure Virtual Machines, Google Compute Engine.
PaaS	Azure App Service, Google App Engine, AWS Elastic Beanstalk, Heroku.
SaaS	Microsoft 365, Google Workspace, Salesforce, Moodle Cloud.

TASK 7: Examination of XaaS (Anything-as-a-Service)

7.1 What XaaS Is and Its Variants

XaaS (Anything-as-a-Service) is the general term for delivering any IT capability, tool or function as an on-demand, subscription-based or pay-per-use service over the internet, extending the IaaS, PaaS and SaaS models. It lets organisations consume specialised functions without building and operating them.

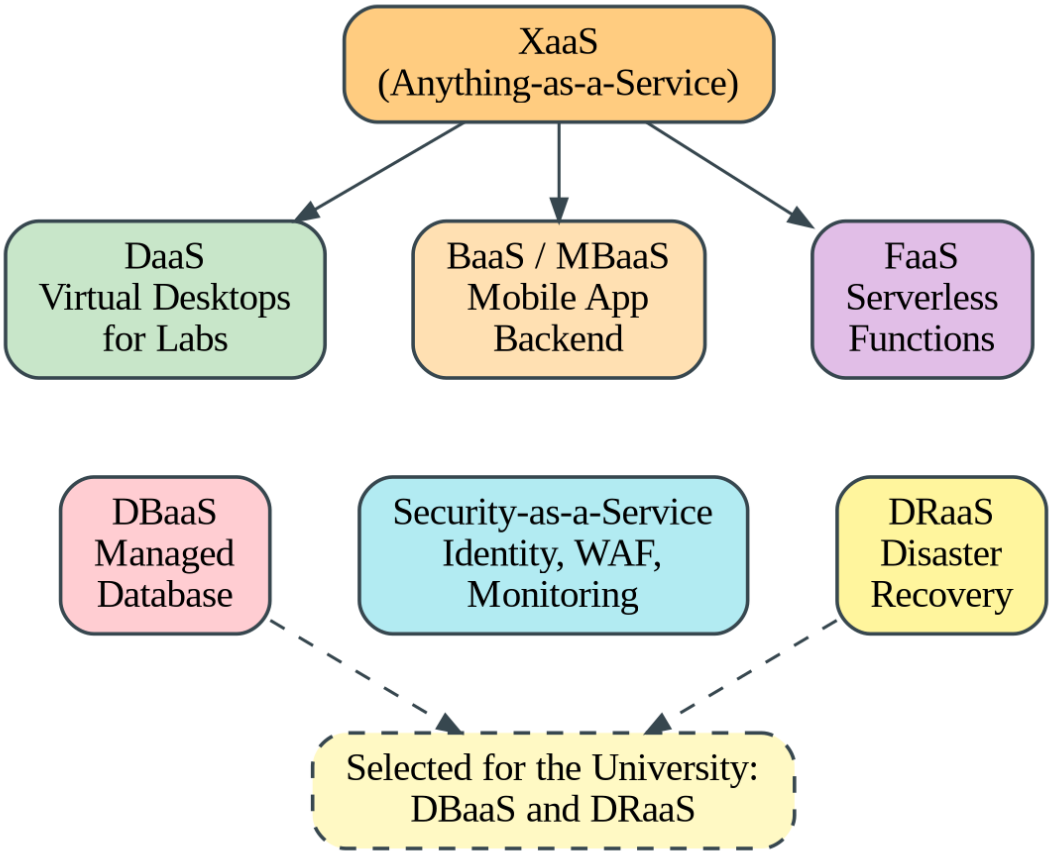


Figure 6: XaaS Variants and the Two Offerings Selected for the University

Table 22: Six XaaS Variants and Their University Use

Variant	Description	Possible Use in the University
DaaS (Desktop as a Service)	Virtual desktops hosted in the cloud and streamed to any device.	Virtual computer labs where students use licensed software from their own devices.

Variant	Description	Possible Use in the University
BaaS / MBaaS (Backend / Mobile Backend as a Service)	Ready backend functions for apps: authentication, database, push notifications and file storage.	Backend of the planned mobile app (login, notifications).
FaaS (Function as a Service)	Small event-driven functions run on demand without managing servers, billed per execution.	Generating fee receipts, sending result notifications, handling peak bursts.
DBaaS (Database as a Service)	A managed database with automatic backups, patching, replication and scaling.	Student, fee and exam records.
Security as a Service	Security functions delivered from the cloud (identity management, web application firewall, monitoring).	Single sign-on and protection of the public web services.
DRaaS (Disaster Recovery as a Service)	Replication of systems and data to the cloud with managed failover and recovery.	Providing the missing disaster recovery site.

7.2 Two Selected XaaS Offerings and Justification

Selection 1: Database as a Service (DBaaS).

- The student, fee and exam records are the university's most valuable data, and the database is the usual bottleneck during admissions and results.
- DBaaS provides automated backups, point-in-time recovery, patching, multi-zone high availability and read replicas, which reduces the workload on a small IT team.
- It scales storage and capacity as the number of users grows, and it encrypts data at rest and in transit.
- It supports the API layer and the future mobile app with predictable performance.

Selection 2: Disaster Recovery as a Service (DRaaS).

- The university has no disaster recovery site at present, so a fire, flood or hardware failure in the single data centre would stop all services.
- Building a second data centre is expensive, whereas DRaaS keeps a replica in the cloud and the university pays mainly for storage until a disaster occurs.

- Recovery time and recovery point targets (for example, 4 hours and 15 minutes) can be agreed in the service agreement and tested regularly without disturbing production.
- It directly addresses a stated problem in the scenario and is a prerequisite for the later 99.9 percent availability requirement.

7.3 Risks of Over-Reliance on Multiple 'as-a-Service' Providers

Table 23: Risks of Depending on Many Service Providers

Risk	**Explanation**	**Mitigation**
Vendor lock-in	Proprietary services and formats make it costly to leave a provider.	Prefer open standards and portable formats, keep data export procedures, and document exit plans.
Integration complexity	Many services from different providers must be integrated, and each has its own interfaces, accounts and security model.	Central identity and API gateway, standard integration patterns.
Cascading outages	An outage at one provider may break dependent services.	Redundancy, monitoring and fallback options for critical services.
Cost creep	Many subscriptions and usage-based charges accumulate and are difficult to forecast.	Tagging, budgets, alerts and regular cost reviews.
Security and compliance gaps	Data is spread across several providers with different controls and locations.	Vendor risk assessment, contracts with data protection terms, encryption and audit logs.
Loss of control and skills	Dependence on external SLAs and roadmaps, and staff may lose technical depth.	Strong SLAs, vendor management roles and continuous training.

TASK 8: Design of the Cloud Solution Architecture

8.1 Architecture Diagram

The proposed architecture combines virtualization, web services and the chosen service models. Users reach the system through a content delivery network and a web application firewall. A load balancer distributes requests to an auto-scaling web and application tier. The tier calls a REST API layer, which uses an in-memory cache, a managed database and object storage. Standard functions run as SaaS. A second region holds replicated data for disaster recovery.

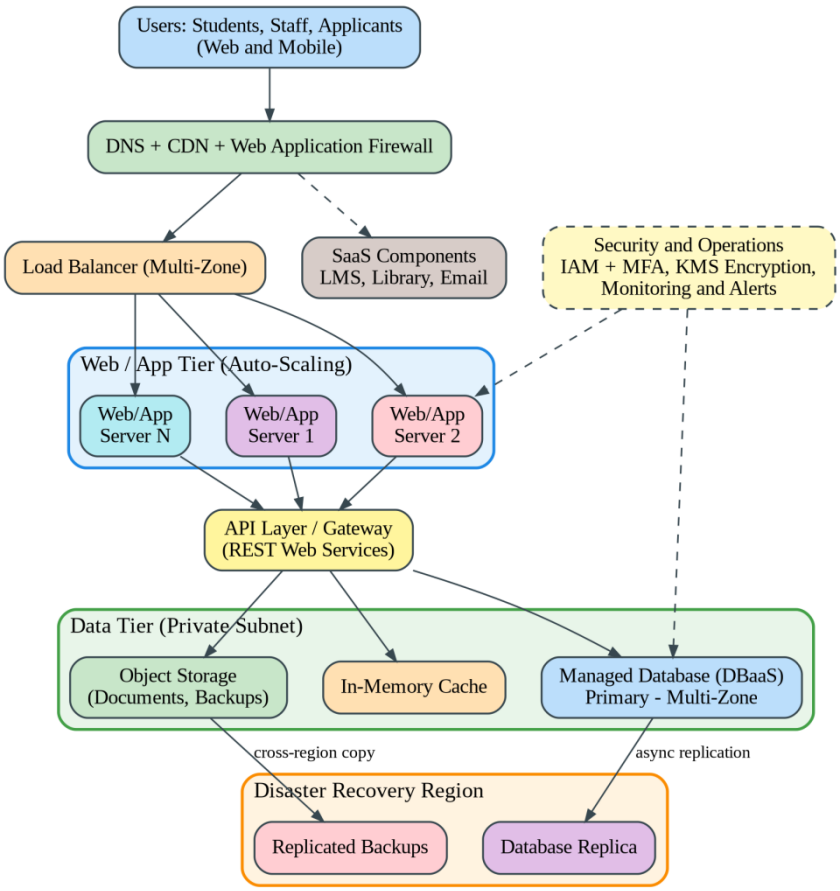


Figure 7: Proposed Cloud Architecture for the University Management System

8.2 Deployment Model and Service Model for Each Component

Table 24: Deployment Model, Service Model and Justification for Each Component

Component	Deployment Model	Service Model	Justification
DNS, CDN and web application firewall	Public cloud	Security as a Service / PaaS	Speeds up content delivery and blocks common web attacks and floods of requests at the edge.
Load balancer	Public cloud	PaaS (managed)	Spreads traffic across zones and removes unhealthy servers automatically.
Web and application tier	Public cloud	IaaS (VMs and containers) with auto-scaling for the Exam Portal, PaaS for the SIS	Elastic capacity for peaks and the control needed for the Exam Portal.
API layer / gateway	Public cloud	PaaS	Single, secured entry point for the mobile app and partner colleges, with authentication and rate limits.
Database tier	Public cloud (in-country region); legacy database remains private during migration (hybrid)	DBaaS	Managed backup, patching, multi-zone failover and replicas.
Object storage	Public cloud	IaaS storage service	Durable and low-cost storage for documents, results archives and backups.
LMS, Library, Email	Public cloud	SaaS	Standard functions with no server maintenance.
Disaster recovery region	Public cloud (second region)	DRaaS	Replica of the database and backups, so services can be restored after a regional failure.

Component	Deployment Model	Service Model	Justification
Identity and access	Hybrid (directory federated with the cloud)	Security as a Service	Single sign-on and multi-factor authentication for all applications.

8.3 Elasticity: Handling Admissions and Exam Peaks

- The web and application servers are placed in an **auto-scaling group** with a minimum of 2 and a maximum of, for example, 20 instances across three availability zones.
- **Target-tracking rules** add instances when average CPU exceeds 60 percent or when requests per instance exceed the limit, and remove them when the load falls.
- **Scheduled scaling** raises the minimum capacity before known events such as admissions, result publication and exam weeks, so servers are ready before the surge.
- The **cache and CDN** absorb repeated requests for results pages and static files, and a **queue** can buffer burst writes such as registration submissions.
- The managed database uses **read replicas** for read-heavy periods and scales storage and capacity as required.
- Monitoring alarms notify the team and provide the data for later tuning and cost review.

8.4 Security and Availability Measures

Table 25: Security and Availability Measures in the Design

Measure	Implementation
Access control	Single sign-on with multi-factor authentication, role-based access control with least privilege, separate roles for students, staff, administrators and partners.
Encryption	TLS for all traffic, AES-256 encryption at rest for databases and storage, keys held in a key management service.
Network security	Private subnets for databases, security groups and firewalls, web application firewall and DDoS protection.
Backup	Automated daily snapshots and point-in-time recovery for the database, versioned object storage, backups kept in a second region.
Multi-zone deployment	Application servers and databases span three availability zones, so one zone failure does not stop the service.

Measure	Implementation
Disaster recovery	Cross-region replication with target recovery time of 4 hours and recovery point of 15 minutes, with a documented, regularly tested failover plan.
Monitoring and audit	Central logging, alerts and audit trails for compliance.

TASK 9: Hands-on Implementation and Documentation

9.1 Objective and Environment

The objective is to build a small proof-of-concept of the design: a containerised REST web service for one university function (result lookup), tested with a client, and then deployed to a cloud platform (PaaS) so that it is reachable through a public HTTPS address. This demonstrates virtualization (container), web services (REST/JSON) and a PaaS deployment, as proposed in Tasks 4 to 8.

Table 26: Environment Used for the Proof-of-Concept

Item	**Details**
Computer	Windows laptop with 8 GB RAM or more (virtualization enabled in BIOS)
Container platform	Docker Desktop (with WSL 2 backend)
Language and framework	Python 3.11 with Flask (REST API) and Gunicorn (production web server)
Test clients	Web browser and curl (curl.exe in Windows PowerShell); Postman can also be used
Cloud platform (PaaS)	Render free tier (or Google Cloud Run / Azure App Service free tier)
Source control	Git and a GitHub repository

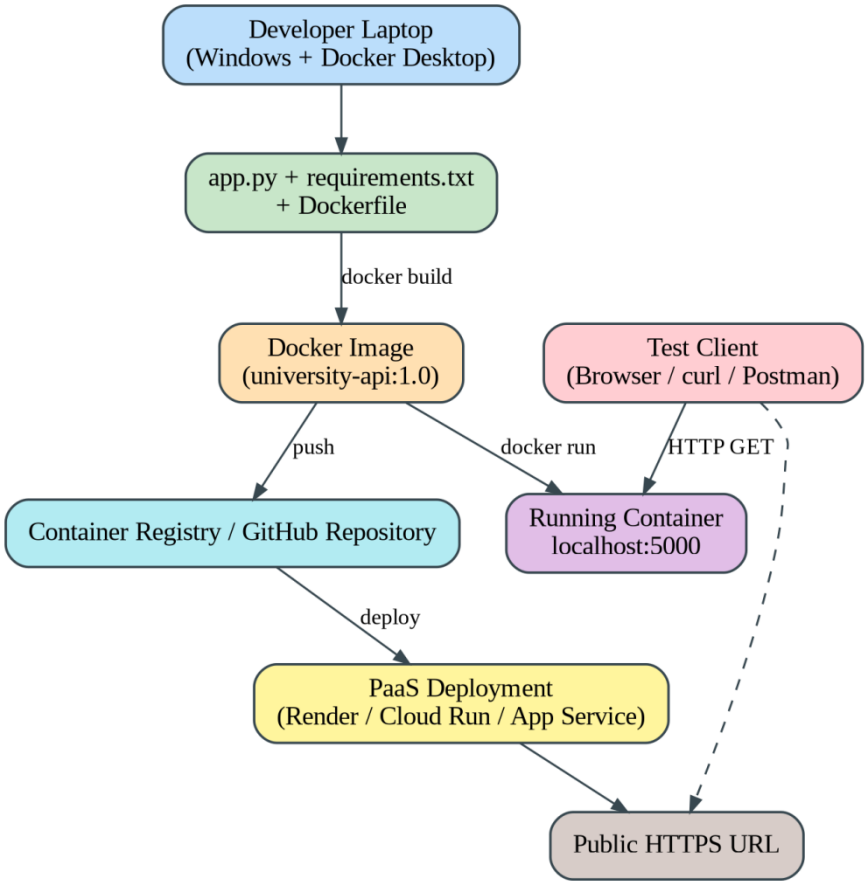


Figure 8: Proof-of-Concept Workflow from Source Code to a Running Container and PaaS Deployment

9.2 Source Code

Create a folder named **university-api** and add the three files below. The file **app.py** defines three routes: the home page, the result lookup service (GET /api/v1/students/{regno}/results) and a health check. An unknown register number returns HTTP 404 with a JSON error message, which is the failure case.

File 1: app.py

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# Sample in-memory data (a real system would read from a managed
database)
RESULTS = {
```

```
    "SIMATS1001": {"name": "Anitha R", "semester": 5, "cgpa": 8.7,
"status": "Pass"},
    "SIMATS1002": {"name": "Karthik S", "semester": 5, "cgpa": 7.9,
"status": "Pass"},
}

@app.route("/")
def home():
    return jsonify({"service": "University Result API", "status":
"running"})

@app.route("/api/v1/students/<regno>/results", methods=["GET"])
def get_result(regno):
    student = RESULTS.get(regno)
    if student is None:
        return jsonify({"error": "Student not found", "regno": regno}),
404
    return jsonify({"regno": regno, **student}), 200

@app.route("/health")
def health():
    return jsonify({"status": "ok"}), 200

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

File 2: requirements.txt

```
flask==3.0.3
gunicorn==22.0.0
```

File 3: Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY app.py .
EXPOSE 5000
CMD ["gunicorn", "--bind", "0.0.0.0:5000", "app:app"]
```

[Screenshot 1: the university-api folder showing app.py, requirements.txt and Dockerfile]

Figure 9: Project Folder Containing the Source Files (Insert Your Own Screenshot Here)

9.3 Creating and Running the Container (Virtualization)

Open Windows PowerShell in the university-api folder and run the following commands.

```
docker build -t university-api:1.0 .
docker run -d -p 5000:5000 --name uni-api university-api:1.0
docker ps
docker logs uni-api
```

- **docker build** reads the Dockerfile and creates an image named university-api version 1.0.
- **docker run** starts a container from the image in the background (-d) and maps port 5000 of the container to port 5000 of the computer (-p).
- **docker ps** lists running containers. The container uni-api should be shown with status Up.
- **docker logs** shows the output of the web server, including the requests received.

[Screenshot 2: output of docker build showing successful completion]

Figure 10: Docker Image Build Output (Insert Your Own Screenshot Here)

[Screenshot 3: output of docker run and docker ps showing the container uni-api in Up status]

Figure 11: Running Container Shown by docker ps (Insert Your Own Screenshot Here)

9.4 Testing the Web Service

The service is tested with a browser and curl. The tests cover the normal case (HTTP 200) and the failure case (HTTP 404).

```
curl.exe -i http://localhost:5000/api/v1/students/SIMATS1001/results
curl.exe -i http://localhost:5000/api/v1/students/XYZ999/results
```

Table 27: Test Cases and Expected Results

Test	Request	Expected Status	Expected Output (Similar To)
Functional test (valid student)	GET /api/v1/students/SIMATS1001/results	200 OK	{"cgpa":8.7,"name":"Anitha R","regno":"SIMATS1001","semester":5,"status":"Pass"}
Failure test (unknown student)	GET /api/v1/students/XYZ999/results	404 Not Found	{"error":"Student not found","regno":"XYZ999"}
Health check	GET /health	200 OK	{"status":"ok"}

[Screenshot 4: curl or browser output for SIMATS1001 showing HTTP 200 and the JSON result]

Figure 12: Functional Test Showing HTTP 200 and JSON Output (Insert Your Own Screenshot Here)

[Screenshot 5: curl output for XYZ999 showing HTTP 404 and the error message]

Figure 13: Failure Case Showing HTTP 404 (Insert Your Own Screenshot Here)

9.5 Deploying to a PaaS Platform

To demonstrate a PaaS component, the same service is deployed to a managed platform, where the provider handles the servers, operating system, scaling and HTTPS certificate. The steps for Render are listed below. The screens of other providers differ slightly, so follow the current screens of the provider you choose.

- Create a GitHub repository and upload the three files (git init, git add ., git commit, git push).

- Create a free account on the PaaS provider and choose to create a new Web Service from the GitHub repository.
- Select the Docker runtime (the Dockerfile is detected), or the Python runtime with the build command pip install -r requirements.txt and the start command gunicorn app:app.
- Select the free instance type and start the deployment. Wait until the status shows Live.
- Open the public HTTPS address provided and repeat the tests of Section 9.4 using this address in place of localhost:5000.

[Screenshot 6: PaaS dashboard showing the service with status Live and its public URL]

Figure 14: PaaS Deployment Dashboard Showing the Live Service (Insert Your Own Screenshot Here)

[Screenshot 7: browser or curl test of the public URL returning the JSON result]

Figure 15: Test of the Deployed Service through the Public HTTPS URL (Insert Your Own Screenshot Here)

9.6 Issues Faced and Resolutions

The following are common issues in this exercise and their solutions. Keep the ones you actually met, and describe them in your own words.

Table 28: Common Issues and Resolutions

Issue	**Cause**	**Resolution**
Docker Desktop does not start	Virtualization is disabled in BIOS or WSL 2 is not installed.	Enable virtualization in BIOS, install WSL 2 and restart Docker Desktop.
Port 5000 already in use	Another program is using the port.	Use another host port, for example docker run -p 5001:5000, and call localhost:5001.
curl shows a PowerShell error	In PowerShell, curl is an alias for another command.	Use curl.exe, or use a browser or Postman.
Cannot connect to localhost	Container not running or the app listens only on 127.0.0.1.	Check docker ps and logs, and make sure the app uses host 0.0.0.0.

Issue	Cause	Resolution
PaaS build fails	Missing requirements file or wrong start command.	Check the build logs, file names and the start command (gunicorn app:app).
First request on the free PaaS is slow	Free instances sleep when idle.	Wait about a minute for the service to wake up and test again.

Reproducibility: Anyone can reproduce the result by creating the three files, running the build and run commands in Section 9.3, and calling the endpoints listed in Section 9.4. The Dockerfile pins the Python version and the requirements file pins the library versions, so the environment is identical on every machine, which shows the portability benefit of containers.

TASK 10: Evaluation and Recommendation of the Final Solution

10.1 Comparison of the Three Options

Table 29: Comparison of On-Premises, IaaS-Based and PaaS/SaaS-Based Options

Criterion	**On-Premises (Modernised)**	**IaaS-Based (Lift and Shift to VMs)**	**PaaS/SaaS-Based (Recommended)**
Cost model	High CapEx for servers, storage and DR site, with fixed running costs.	No CapEx, OpEx for VMs and storage; VMs run all year unless scheduled.	No CapEx; subscription and usage-based OpEx; lowest operational staff cost.
Scalability	Limited to purchased hardware; 6 to 8 weeks to add capacity.	Good, with auto-scaling, but the university must design and manage it.	Very good: platform and SaaS scale largely automatically.
Availability	Single site unless a second data centre is built.	Multi-zone and multi-region possible but built by the university.	High, with provider-managed redundancy and SLAs.
Security	Full control but the university carries all responsibility and cost of protection.	Shared responsibility; the university secures OS and applications.	Provider secures most layers; the university manages identity, data and configuration.
Management effort	Highest.	High: patching, OS and middleware.	Lowest.
Time to deploy	Slowest.	Medium.	Fastest for SaaS; medium for refactored SIS.
Flexibility and control	Highest.	High.	Lower, with some vendor lock-in.

10.2 Three-Year Cost Comparison

The estimate uses the following assumptions. All amounts are in Indian rupees (lakh = 100,000) and are planning estimates, which must be confirmed with the pricing calculators of the chosen providers and education discounts.

- Planning period of 3 years, with no inflation and no discounting, to keep the comparison simple.

- On-premises: 8 new servers at 4.5 lakh each (36 lakh), storage and network 14 lakh, and a small DR site 25 lakh as capital expenditure in year 1. Annual costs: power, cooling and space 9 lakh, maintenance contracts 7 lakh, 4 administrators 24 lakh and software licences 10 lakh.
- IaaS-based: one-time migration cost of 10 lakh. Annual costs: compute 26 lakh, storage, backup and DR 7 lakh, network and data transfer 3 lakh, licences 8 lakh and 3 administrators 18 lakh.
- PaaS/SaaS-based: one-time migration, refactoring and training of 20 lakh. Annual costs: SaaS subscriptions (LMS, library, email) 14 lakh, PaaS and DBaaS (SIS) 17 lakh, IaaS for the Exam Portal 6 lakh, DR and network 6 lakh and 2 administrators 12 lakh.

Table 30: Three-Year Cost Estimate (Rs. Lakh)

Cost Item	**On-Premises**	**IaaS-Based**	**PaaS/SaaS-Based**
Capital / one-time cost	75	10	20
Annual operating cost	50	62	55
Operating cost for 3 years	150	186	165
Total 3-year cost	225	196	185
Cash outlay in year 1	125	72	75
Saving compared with on-premises	Not applicable	29 (12.9%)	40 (17.8%)

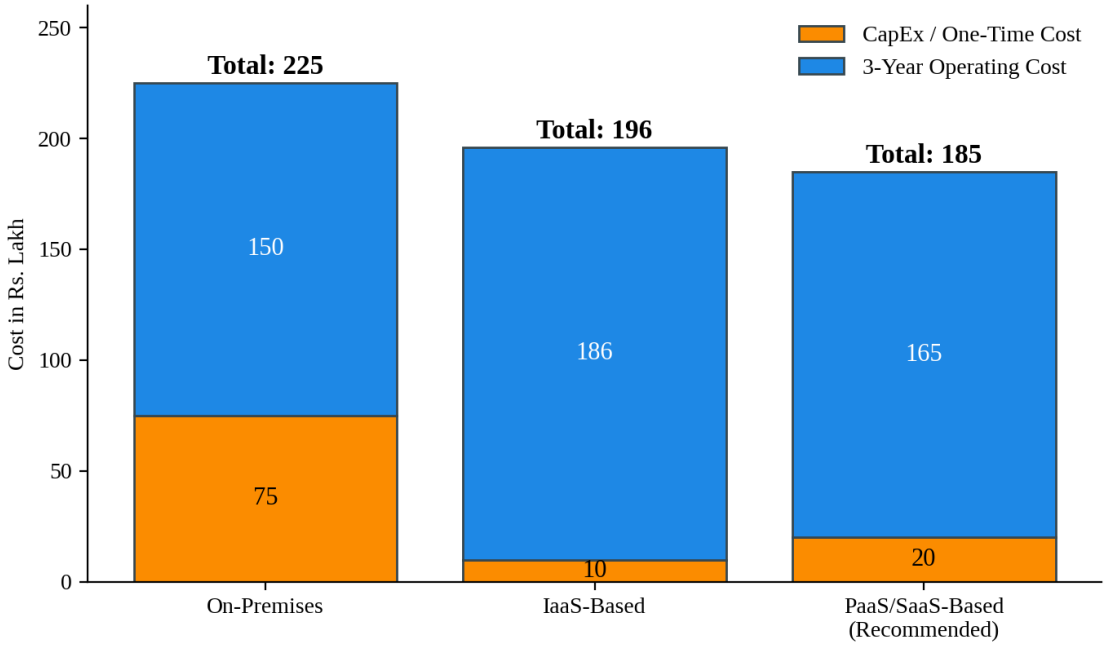


Figure 16: Three-Year Cost Comparison of the Three Options (Rs. Lakh)

The PaaS/SaaS-based option has the lowest three-year cost and the lowest staff cost, and it provides elasticity and disaster recovery that the on-premises option would need additional capital to match. The on-premises figure also excludes the cost of unused capacity bought for peaks, which makes it even less attractive.

10.3 Main Risks and Mitigation

Table 31: Risks of the Proposed Cloud Solution and Mitigation Steps

Risk	**Description**	**Mitigation**
Vendor lock-in	Dependence on one provider's platform and formats makes leaving costly.	Use open standards (containers, REST, SQL), keep data exports and infrastructure-as-code scripts, and choose portable services where possible.
Data privacy and compliance	Student data is personal and subject to data protection law, and may need to stay in the country.	Choose in-country regions, encrypt data, restrict access, keep audit logs and sign data processing agreements.
Downtime	Provider outages or migration errors interrupt admissions or exams.	Multi-zone deployment, DRaaS, tested rollback plans, migration outside peak periods, and SLA review.

Risk	Description	Mitigation
Skills gap	Staff lack experience in cloud operations and security.	Training and certification, a cloud centre of excellence, and support from the provider or partner during the first year.
Cost overruns	Usage-based billing can exceed forecasts.	Budgets and alerts, tagging, rightsizing, reserved capacity for baseline load and regular reviews.
Migration failure	Monolithic applications may not move cleanly.	Phased migration, pilot, data validation and parallel run before cutover.

10.4 Final Recommendation and Migration Roadmap

Recommendation: The university should adopt the **PaaS/SaaS-led hybrid cloud solution**: standard functions (email, library, LMS) move to SaaS, the Student Information System moves to a managed platform with a managed database (PaaS and DBaaS), and the Exam Portal runs on auto-scaling IaaS (virtual machines or containers). Disaster recovery is provided by a second region (DRaaS). The recommendation is based on the analysis in earlier tasks: it removes the idle capacity identified in Task 1, uses the consolidation and the container decisions of Task 4, follows the application mapping of Task 6, uses the DBaaS and DRaaS choices of Task 7, and has the lowest three-year cost in Section 10.2. It was selected on its fit with the university's needs and not on the popularity of a provider; any provider that meets the in-country data location, security and price requirements can be used after a proof-of-concept and price comparison.

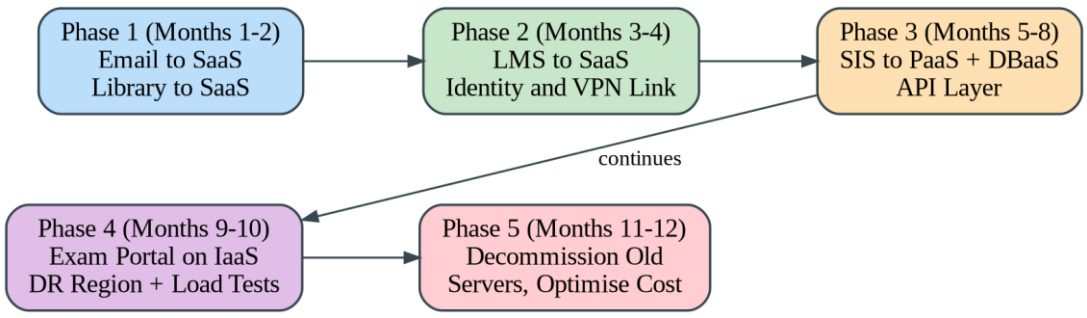


Figure 17: Phased Migration Roadmap for the University Applications

Table 32: Migration Roadmap and Reasons

Phase	**Applications**	**Reason for the Order**
Phase 1 (months 1 to 2)	Email and Library move to SaaS.	They are standalone, have no custom code and little data dependency, so the risk is lowest. They give a quick win, free up two servers and build the team's confidence.
Phase 2 (months 3 to 4)	LMS moves to SaaS; identity federation and the secure link are set up.	The LMS is large but standard. Single sign-on and the VPN prepared here are needed by the later phases.
Phase 3 (months 5 to 8)	SIS moves to PaaS with DBaaS; the API layer is added.	It is the most complex and critical system, so it is moved after the team has gained experience, with data replication and a parallel run.
Phase 4 (months 9 to 10)	Exam Portal moves to auto-scaling IaaS; the DR region is activated; load tests are run.	It has the highest peaks, so it is completed with full load testing before the next examination cycle.
Phase 5 (months 11 to 12)	Old servers are decommissioned and costs are optimised.	Old hardware is retired only after stable operation, to keep a fallback until the end.

TASK 11: Day-4 Design Challenge: Re-evaluation and Adaptation of the Design

Decision Summary

The original design is **modified**, not completely redesigned. Its main principles remain valid: a hybrid, cloud-first approach, SaaS for standard functions, managed database and disaster recovery. However, to serve a mobile app and 40 affiliated colleges with 10 times the peak load and 99.9 percent availability within a limited budget, the design is strengthened with an API gateway, a container platform, serverless functions, multi-zone deployment of all tiers, caching, and tenant isolation. The reasons are given in the four sections below.

11.1 Impact Analysis

What has changed: The system was designed for internal use by about 27,000 students and staff on 4 campuses, mostly through the campus network. It must now serve a public mobile app and 40 external colleges over the internet, with 10 times the peak load, a 99.9 percent availability target and a limited operating budget.

Table 33: Original Assumptions Affected by the Changed Requirement

Assumption	Original Design	Changed Requirement	Effect
Load	Assumed peak of about 150 requests per second (estimate)	10 times the peak, about 1,500 requests per second	Capacity, caching and database design must scale much further.
Users	About 27,000 (25,000 students and 2,000 staff)	Adds the mobile app users and 40 colleges (about 60,000 more users, assuming 1,500 students per college)	About 87,000 users, with external and unpredictable access.
Availability	No stated target; single region, approximately 99.5 to 99.7 percent	99.9 percent, about 8.76 hours downtime per year	Multi-zone design of every tier and tested failover are needed.
Cost	Estimated 55 lakh per year	Limited operating budget	Linear scaling is unaffordable; elasticity and cost control are essential.
Access	Internal users	External apps and organisations	Security, authentication and isolation become critical.

Why web services and APIs now become critical: The mobile app and the colleges' own systems cannot use the university's internal screens. They must call defined, versioned and secured APIs for student records and fee payment. The API layer becomes the single controlled entry point where authentication, throttling, monitoring and versioning are applied, and it allows the back-end to scale and change without breaking the clients.

Effect of multi-tenant access by 40 colleges on security and isolation: Each college is a tenant who must see only its own students and fee data. Risks include data leakage between colleges, one college's heavy use slowing the others (noisy neighbour), stolen credentials and legal obligations for personal data. Therefore tenant identity must be carried in every request, data must be isolated per tenant, and usage limits and audit logs must be applied per tenant.

11.2 Evaluation of the Original Design

Table 34: Evaluation of the Original Design under the New Conditions

Aspect	**Assessment of the Original Design**
Service models and virtualization plan at 10 times load	SaaS components (LMS, email, library) scale at the provider, only the licences and subscription cost grow. The SIS on PaaS can scale, but it is a monolith that scales mostly by running larger instances, which is costly and has limits. The Exam Portal on VMs scales out, but VMs start in minutes and must be sized for the peak. The plan of 4 consolidated hosts supports only the normal load. Verdict: not sufficient without modification.
Capacity needed	At 1,500 requests per second and 50 requests per second per application instance at 70 percent utilisation, 1,500 / 50 = 30 instances are required, against the few instances planned originally.
Auto-scaling and single-region deployment versus 99.9 percent	If a single-zone chain of three components (load balancer, application tier, database) each has 99.9 percent availability, the combined availability is 0.999 x 0.999 x 0.999 = 0.997 (99.7 percent), that is 8,760 x 0.003 = about 26.3 hours of downtime per year, which is more than the 8.76 hours allowed. Auto-scaling alone does not protect against a zone or region failure. Verdict: insufficient.
Cost under the new load	If the original design were simply scaled ten times all year, its compute cost (23 lakh) would rise to 230 lakh, and the annual total to about 262 lakh (230 + 32 lakh of other costs). This is not acceptable for a limited budget.

Aspect	Assessment of the Original Design
Better service model or XaaS option	Yes. An API gateway (PaaS), container orchestration (container as a service), serverless functions (FaaS) for bursty tasks such as receipts and notifications, a CDN and distributed cache, and a managed message queue provide better scalability per rupee. Mobile backend as a service (MBaaS) functions could also be used for push notifications and authentication.

11.3 Adaptation or Redesign

Decision: MODIFY. A complete redesign is not needed because the SaaS applications, DBaaS and DRaaS choices, identity federation and phased roadmap remain valid, and replacing everything would increase cost and risk. The following modifications are made.

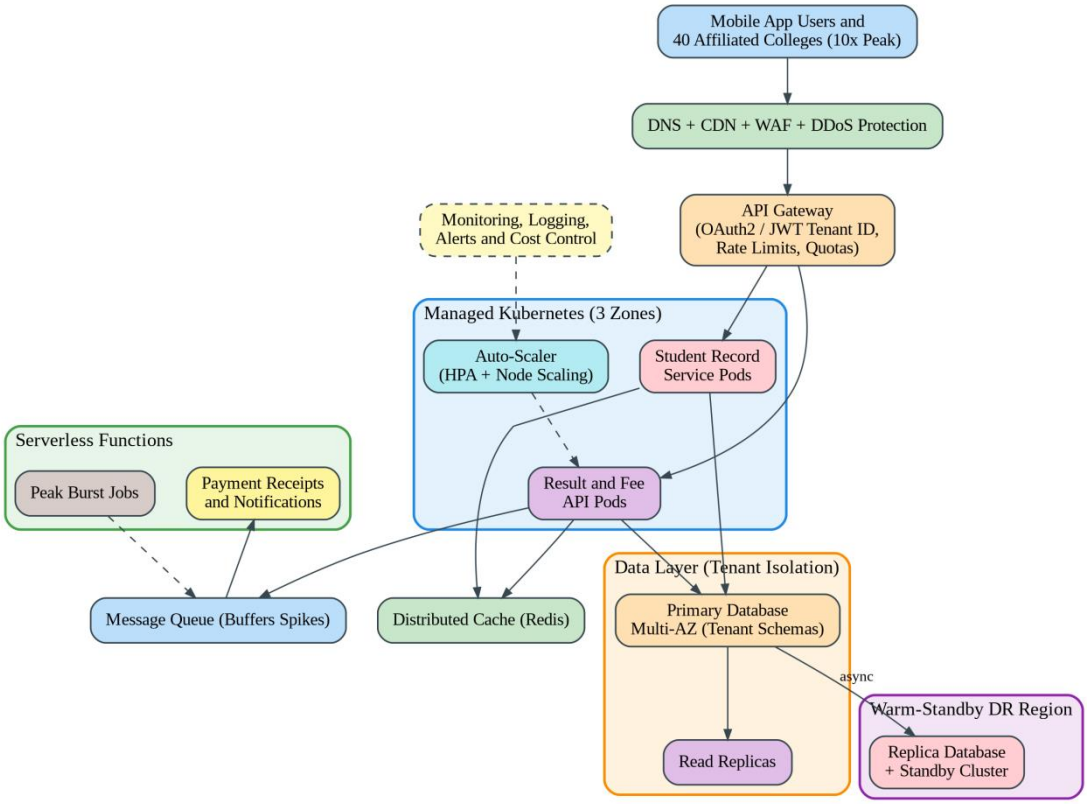


Figure 18: Revised Architecture for the Mobile App and 40 Affiliated Colleges

Table 35: Changes to the Service Model Mix and Virtualization Approach

Area	Original Design	Revised Design
Entry point	Load balancer and web servers.	CDN, WAF and DDoS protection, with an API gateway handling authentication, tenant identification, rate limits and quotas.
Compute and virtualization	Consolidated VMs and some containers on 4 hosts, with VM-based auto-scaling for the Exam Portal.	Containers on a managed Kubernetes cluster across 3 zones with horizontal pod and node auto-scaling; VMs only for legacy parts. Containers start in seconds, which suits 10 times peaks.
Serverless	Not used.	Functions for receipts, notifications and burst jobs, billed per execution, fed by a queue that buffers spikes.
Data layer	DBaaS with one primary.	Multi-zone DBaaS with read replicas, a distributed cache, per-tenant schemas, and a warm-standby replica in a second region.
Service model mix	SaaS for LMS, library, email; PaaS for SIS; IaaS for the Exam Portal.	More PaaS and container-based services plus FaaS; SaaS unchanged; IaaS reduced to special cases.
Availability	Single region, DR through restore.	Multi-zone for every tier, health-checked failover and warm-standby DR region with tested failover.
Mobile support	Not available.	Versioned REST APIs, push notifications and MBaaS-style authentication.

11.4 Evidence-Based Defence

(a) Capacity calculation

```
Peak load = 10 x 150 = 1,500 requests per second (assumed baseline peak
of 150)
Instances needed = 1,500 / 50 = 30 instances
To survive the loss of one of 3 zones at full peak, the remaining 2
zones must carry 30 instances:
30 / 2 = 15 per zone, so 3 x 15 = 45 instances at the peak only
Normal load needs about 6 to 8 instances; auto-scaling provides the rest
only during peaks
```

With caching at a 70 percent hit ratio, reads of about 1,200 requests per second (assuming 80 percent reads) fall to 360 per second at the database. With 300 writes per second, the database handles about 660 operations per second, spread across the primary and two read replicas. The CDN further removes static and repeated content from the origin.

(b) Availability calculation

```
Multi-zone components: load balancer 99.99%, application tier 99.99% (3
zones), database 99.95% (multi-zone)
Combined availability = 0.9999 x 0.9999 x 0.9995 = 0.9993 (99.93%)
Downtime = 8,760 hours x (1 - 0.9993) = about 6.1 hours per year
Target: 8.76 hours per year, so the design meets 99.9%
Comparison: single-zone design = 99.7% = about 26.3 hours per year
(fails)
```

These figures use typical published service levels and are design estimates. Actual availability must be validated with failover tests and the providers' agreements.

(c) Cost comparison between the original and the revised design

Table 36: Annual Operating Cost Comparison (Rs. Lakh)

Cost Item	**Original Design (Task 10)**	**Original Scaled 10 Times All Year**	**Revised Design**
Compute (PaaS, IaaS, containers, functions)	23	230	83
API gateway, CDN and WAF	0	0	8
Container platform and monitoring	0	0	6
Database (multi-zone, replicas)	Included above	Included above	6
SaaS subscriptions (additional users)	14	14	8
DR and network	6	6	8
Staff	12	12	14
Total per year	55	262	133

The compute figure of the revised design is estimated as 23 x (0.15 x 10 + 0.85 x 2.5) = 83 lakh, assuming that the 10 times load occurs for about 15 percent of the year (admissions,

results and exams), and that the rest of the year the load is about 2.5 times the original because of the mobile app and colleges. The revised design therefore costs about 133 lakh per year, which is 129 lakh (49 percent) less than simply scaling the original design. For three years, the revised design costs about 25 + 3 x 133 = 424 lakh (including a one-time upgrade cost of 25 lakh) against 20 + 3 x 262 = 806 lakh for the scaled original. Further cost controls include reserved capacity for the baseline load, scheduled scaling, CDN caching, serverless billing per use and budget alerts. The university can also recover cost through a chargeback to the affiliated colleges: for example, a fee of Rs. 150 per student per year for 60,000 students gives Rs. 90 lakh per year (illustrative).

(d) Security and multi-tenancy considerations

- **Authentication and authorisation:** OAuth2 with signed tokens (JWT) carrying a tenant identifier, validated by the API gateway on every request; role-based access for college staff, students and administrators.
- **Data isolation:** Separate database schemas (or row-level security) for each college, so one tenant can never query another's data; per-tenant encryption keys for sensitive data.
- **Noisy-neighbour control:** Rate limits and quotas per tenant at the gateway, and resource limits for containers.
- **Transport and perimeter security:** HTTPS everywhere, WAF, DDoS protection and optional private connectivity for large colleges.
- **Monitoring and compliance:** Audit logs per tenant, anomaly alerts, regular access reviews and compliance with the applicable data protection law.

(e) Trade-offs

Table 37: Trade-Offs of the Revised Design

Trade-Off	**Explanation**	**How It Is Managed**
Vendor lock-in	Managed services and serverless functions tie parts of the solution to one provider.	Use Kubernetes, containers, REST and standard SQL; keep infrastructure as code and an exit plan.
Complexity	Containers, an API gateway and serverless functions are more complex to operate than a few VMs.	Use managed services, automation and monitoring, and train the team.

Trade-Off	Explanation	How It Is Managed
Migration effort	Splitting the monolith and re-pointing clients takes time and risks errors.	Apply the strangler approach: move one API at a time, starting with result lookup and fee payment, with rollback plans.
Cost versus resilience	A warm-standby region and multi-zone design cost more than a single zone.	Use scale-to-zero or small standby capacity, and apply the highest standard only to critical services.

Conclusion: The modified design meets the 10 times load through elastic containers, caching and serverless functions, reaches an estimated 99.93 percent availability through multi-zone deployment and a warm-standby region, stays at about half the cost of scaling the original design, and protects the 40 colleges through tenant isolation. The decision therefore follows logically from the calculations above.